



Welcome to

# Exam Project

## Software Security – FlyBook

Henrik Kramselund he/him han/ham hlk@zencurity.com @kramse

Slides are available as PDF, <https://codeberg.org/kramse/exam-project-flybook.tex> in the repo security-courses

© license CC BY 3.0. 2026 Henrik Kramselund



- System that allows the user to search for flights and book them

**FlyBook**  
Book your next flight in less than one minute. Just type the info below and your flight will be booked.

From ✈️: Aalborg, Denmark AAL, 1  
Destination ✈️: Aberdeen, SD ABR, 9  
When 📅: 01-05-2018

**Find**

**Flights ✈️**

| Flight number | From    | Destination | Departure time      | ETA                 | Gate |                      |
|---------------|---------|-------------|---------------------|---------------------|------|----------------------|
| JS1213        | Aalborg | Aberdeen    | 2018-05-01 08:00:48 | 2018-05-02 23:06:38 | A67  | <a href="#">Book</a> |
| JS1213        | Aalborg | Aberdeen    | 2018-05-01 08:00:48 | 2018-05-02 23:06:38 | A67  | <a href="#">Book</a> |
| TA113         | Aalborg | Aberdeen    | 2018-05-01 08:00:48 | 2018-05-02 23:06:38 | B12  | <a href="#">Book</a> |
| JS1213        | Aalborg | Aberdeen    | 2018-05-01 08:00:48 | 2018-05-02 23:06:38 | A67  | <a href="#">Book</a> |
| TA113         | Aalborg | Aberdeen    | 2018-05-01 08:00:48 | 2018-05-02 23:06:38 | B12  | <a href="#">Book</a> |
| SAS113        | Aalborg | Aberdeen    | 2018-05-01 14:00:48 | 2018-05-02 21:09:38 | C2   | <a href="#">Book</a> |

# The task



- You are asked to make a security assessment of the FlyBook system
- Unfortunately, the documentation of the system is not available
- The good news is that you are given access to the source code
- You need to hand in a security assessment report

Source: <https://github.com/sebikea/flybook>

Note: if you have problems running the project work together with other groups!

Recommend cloning the project and making local changes - track changes

# Report – required contents



Can be structured as you wish, but make sure it includes:

Description of the system and what kind of system it is, what is the context – a half page that sets the scene, what kind of security does such a system warrant

## Static analysis (code review)

- Automated code review – use at least two SAST tools  
Describe a little about the settings, maybe include some config in appendices
- Manual code review – have you noticed something bad

## Dynamic analysis (while code is running)

- Automated analysis – use at least one DAST tool – could be dynamic audit/runtime or pentesting  
Describe a little about the settings, maybe include some config in appendices
- Manual pentesting (optional) – try working with the application in Burp  
Document some of the findings – use categories informational, low risk, medium risk or high risk

## Report – required contents, cont.



Security problems and recommendations (possible/suggested fixes)

- Explain what should be changed and how (not detailed code rewrite or refactor)
- You are encouraged to provide references to CWE

Risk analysis to identify the most urgent vulnerabilities to be fixed  
(detailed risk analysis in the appendix)

Priorities - easy fixes, critical fixes, where would you start!

Common report sections: introduction, conclusion,  
references/literature, and appendix (bilag) if needed

## Aspects to consider – tools



- Which tools have you selected and used? Why?
- What tools or testing approaches give you the best results and why?
- If the tool allows it, tune it to have fewer false positives
  - and explain in the report what you did
- **Don't include all findings from the automated code review tools:**  
select the most relevant ones or summarize the results

## Aspects to consider – findings



What are the most serious vulnerabilities and why?

How is data transmitted between the different components?

GET/POST requests, HTTPS?

Code review – remember to look at:

- Errors and exception handling
- Container security best practices
- Logging
- You can also use Software Composition Analysis (SCA) tools (dependency checking)

## Report – formalities



- Has to be written in Danish (unless you have applied for permission)
- Max. 10 A4 pages + 5 pages per student (approx.)
- Group of 2 students: max. 10+10 pages
- Group of 3 students: max. 10+15 pages
- Group of 4 students: max. 10+20 pages
- Do your best to work in groups of 2–4 students
  - please try to avoid 1 person groups as it will be too much work!

# Exam project hand-in



- Through WiseFlow
- Deadline: TBD, I will check with Dany
- Remember to include the names of all group members in the report
- Report should be in PDF
- If you modify the code to fix the vulnerabilities (not mandatory)  
you may include ZIP or reference your Git repository

# Exam



- Dates: TBD – I think they are scheduled already!
- Oral, individual exam
- Language: Danish
- 10 min presentation of the findings  
+ 15 min questions based on the project and course curriculum
- 7-point grading scale

## Presentation at the exam



Not mandatory to use slides, but they can provide structure  
(try to prepare different slides per group member)

Max 10 min – you can summarize the findings

You can include extra bits not covered in the report but seen in the course:

- Software security touchpoints
- Security design principles
- ...

Make sure you understand at a high level the vulnerabilities  
you mention in your presentation – what are the consequences of these

You can do a demo of an attack – not recommended, takes time!